

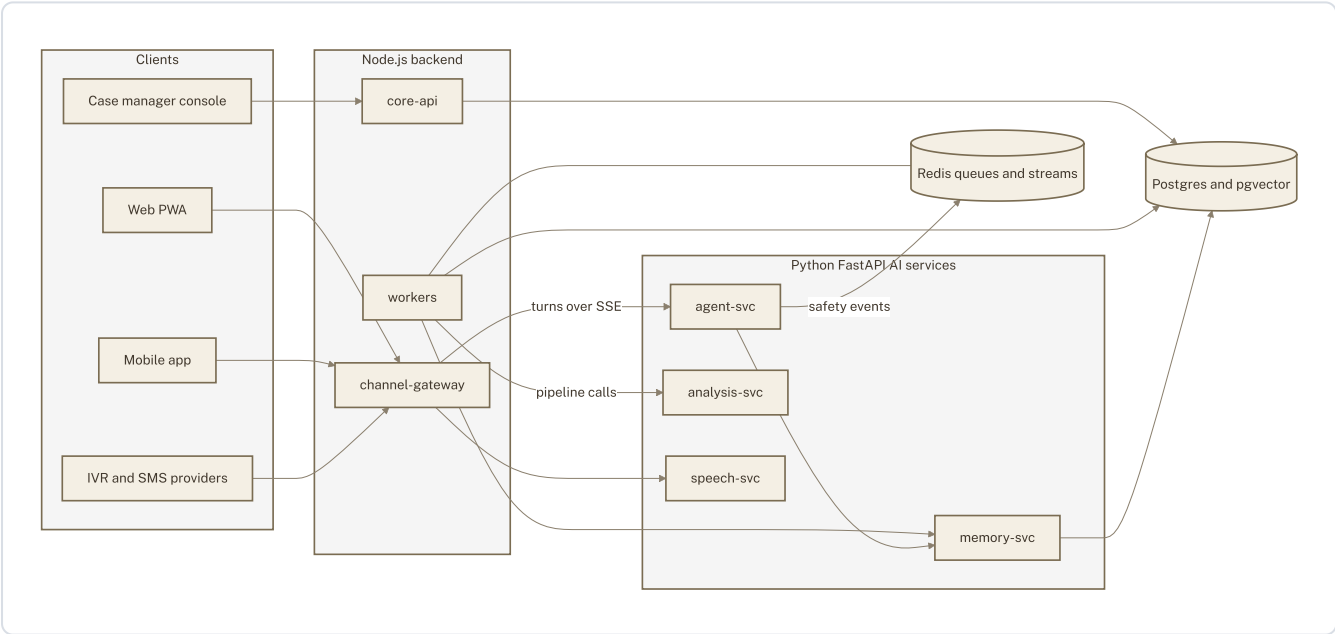
Aavaz technical workflow

End-to-end specification of how a survivor moves through Aavaz: which service does what, in what order, with which data, events, timers and safety controls.

Draft v0.2 for team review, September 2026. Backend in Node.js (JavaScript); LLM and AI tasks in Python with FastAPI. Statutory references are placeholders for review by a legal advisor before any real use.

1 System map

Aavaz is split by language along one clear line. **Node.js (JavaScript)** owns the product backend: channels, auth and consent, profiles and cases, scheduling, triage, referrals, the console API, and every write to domain tables. **Python (FastAPI)** owns everything model-related: the conversation agent, memory and retrieval, speech, summarisation, assessment and legal detection. Python services contain no business rules about who gets contacted or referred, and Node never calls an LLM directly.



Node.js services

Service	Responsibility	Libraries
channel-gateway	Victim WebSocket sessions, IVR and SMS webhooks, envelope validation, streaming turns to agent-svc and replies back to the victim.	NestJS, Socket.IO, Twilio or Exotel SDK, Zod

core-api	Modular monolith with modules identity-consent , profile-case , triage-router , referral , console , oversight , audit . Sole writer of domain tables.	NestJS, Prisma, argon2, jose, json-rules-engine, Puppeteer (PDF)
workers	Court sync, check-in scheduling, silence ladder, post-session orchestration, SLA timers, notifier, purge job.	BullMQ (MVP), Temporal TypeScript/JavaScript SDK (production), ioredis
console (frontend)	Case manager and supervisor web app.	React, TanStack Query

Python AI services (FastAPI)

Service	Responsibility	Libraries
agent-svc	LangGraph check-in graph, crisis guard, output guard, session state checkpoints, transcript for a session.	FastAPI, LangGraph (Postgres checkpointer), Pydantic, sse-starlette
memory-svc	Context assembly, rolling summaries, chunking, embeddings, victim-scoped retrieval, memory purge.	FastAPI, SQLAlchemy async, pgvector, sentence-transformers (BGE-M3) or Text Embeddings Inference
analysis-svc	Fact extraction, screener scoring, distress classification, safety rules, score fusion, legal issue detection with verified statute citations.	FastAPI, Pydantic structured outputs, LangChain retrievers
speech-svc	Streaming speech recognition, text-to-speech and translation for voice and SMS.	FastAPI, Bhashini API client, optional AI4Bharat models

Data ownership

Data	Writer	Access from the other side
victims, cases, milestones, consents, safety_settings, checkins, sessions, assessments, extracted_facts, tasks, referrals, staff, audit_log	Node core-api and workers via Prisma	Python gets only what a request carries; no direct DB role on these tables
memory_chunks, case_summaries,	Python services	Node reads only through

statute_chunks, langgraph_checkpoints	via SQLAlchemy (Alembic migrations)	memory-svc endpoints
Transcripts, audio, referral PDFs	Node, encrypted in object storage	Python fetches a transcript only during an analysis call, by signed short-lived reference

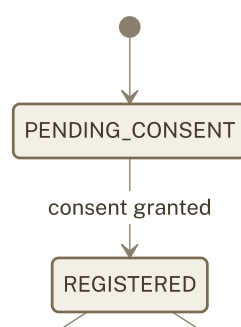
How Node and Python talk

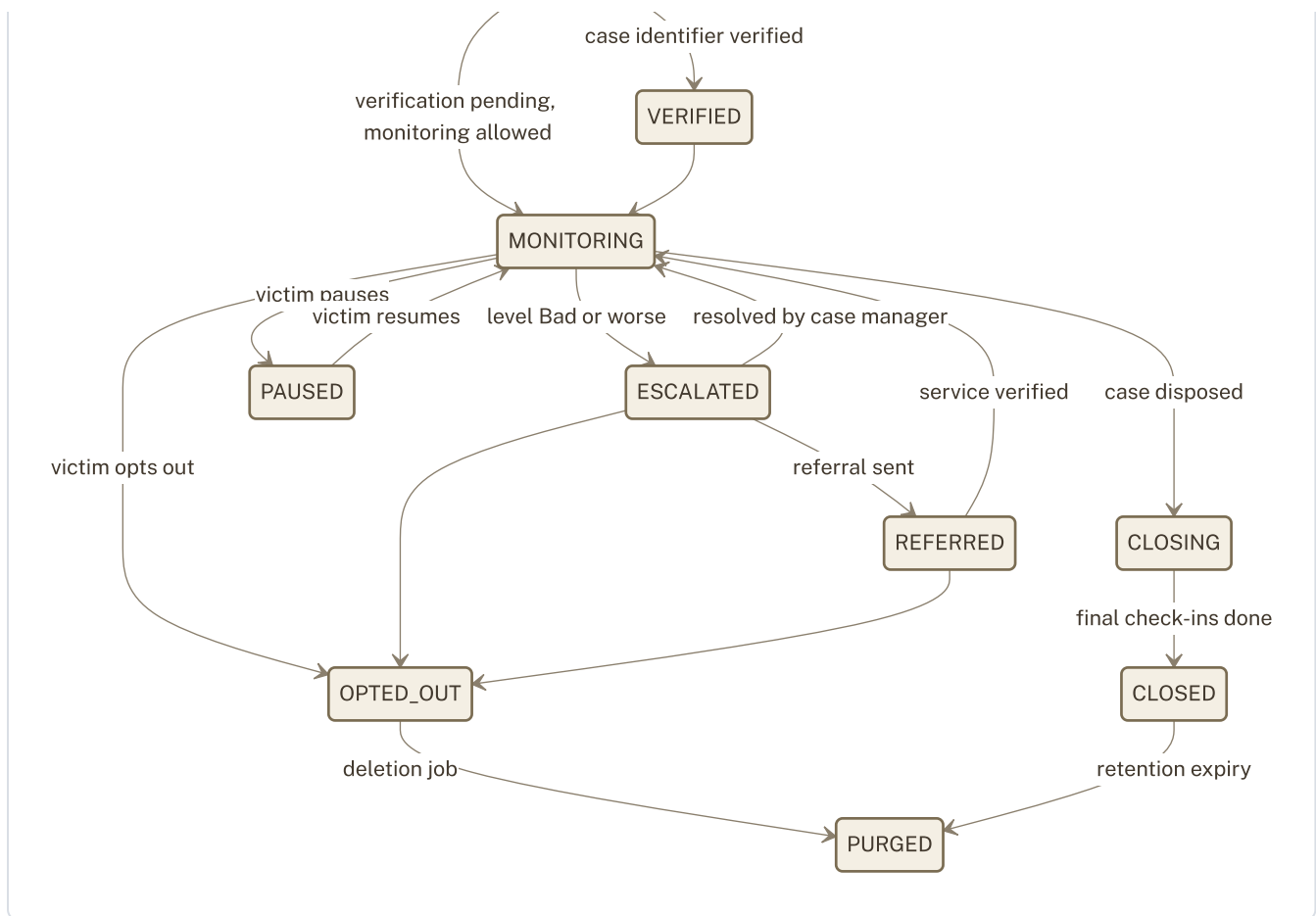
- **Synchronous calls:** internal HTTP/JSON. FastAPI publishes OpenAPI schemas and the Node side generates its client from them (for example with `openapi-typescript` plus `openapi-fetch`), so a contract change breaks the build instead of production.
- **Streaming:** each conversation turn is a `POST` to `agent-svc` that returns a server-sent event stream of `token`, `safety` and `done` events, which the gateway relays to the victim.
- **Events:** Redis Streams. Node publishes and consumes most topics with `ioredis`; Python publishes only `safety.crisis_detected` and `session.ended` with `redis-py` `asyncio`.
- **Service auth:** `core-api` issues short-lived service JWTs (5 minutes, audience per service) with `jose`; FastAPI verifies them in a dependency. Production adds mTLS inside the cluster.
- **Timeouts and retries:** conversation turn 20 s, analysis calls 120 s; every call carries an `x-request-id` so retries are idempotent.

Shared infrastructure: PostgreSQL 16 with `pgvector` and row-level security, Redis (sessions, rate limits, BullMQ queues, Redis Streams), object storage for encrypted audio and PDFs, and a KMS for envelope encryption.

2 Case lifecycle

Every victim record carries exactly one lifecycle state. All workflows below are transitions on this machine.

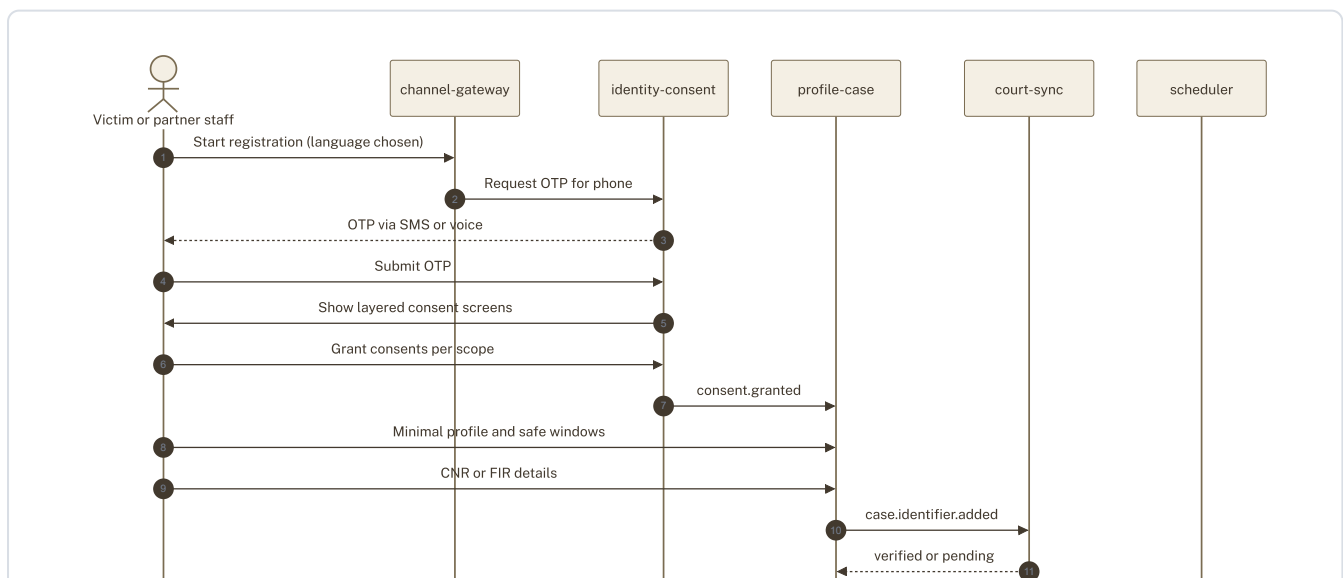


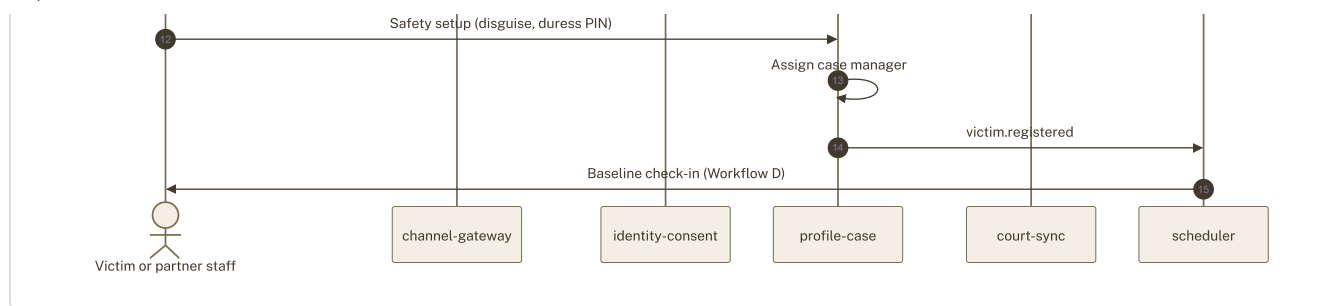


Rules: only MONITORING , ESCALATED , REFERRED and CLOSING may receive outbound check-ins. PAUSED stops scheduled outreach but keeps victim-initiated contact and open referral tracking. OPTED_OUT stops all outbound contact immediately.

3 Workflow A: Onboarding

Two entry paths share one pipeline. Self-onboarding comes from the web portal, app or toll-free line. Assisted onboarding is done by partner staff (One Stop Centre, DLSA front office, NGO) using an org-scoped account, always with the victim present and consenting.





- A1 Entry and age gate.** First question after language: "Are you 18 or older?" If no, stop self-onboarding, show child helpline and One Stop Centre information, and create an unlinked handoff task for a trained case manager. The minor's details are not stored in the adult flow.
-
- A2 Phone verification.** 6-digit OTP, 5-minute expiry, 3 attempts, rate-limited per phone and per IP. IVR reads the OTP aloud to the calling number. Phone is stored as a salted hash for lookup plus an encrypted value for outreach.
-
- A3 Consent capture.** One screen per scope, in the chosen language, with an audio version. Scopes: `monitoring` (required), `store_transcripts`, `voice_recording`, `share_mental_health`, `share_legal_aid`, `share_welfare`, `share_protection`. Each grant writes a row with the SHA-256 hash of the exact consent text version shown.
-
- A4 Minimal profile.** Alias or name, language, district, relation to case (survivor or family member), crime category, preferred channel, safe contact windows (for example weekdays 11:00–13:00), whether calls are safe at all.
-
- A5 Case identifiers.** CNR validated against `^[A-Z]{4}[0-9]{12}$` and queued for verification. FIR number, police station and year stored encrypted and marked for manual verification by the case manager. Registration continues if neither is available.
-
- A6 Safety setup.** Optional app disguise (neutral name and icon), duress PIN (Argon2id hash, must differ from login PIN), optional trusted contact, a safe word the case manager will use on calls, and a quick-exit control explained with a short demo.
-
- A7 Assignment.** Case manager chosen by district, language, and lowest active caseload below the cap (default 80). No match creates an unassigned task for the district supervisor.
-
- A8 Baseline check-in.** Scheduler starts the victim's long-running check-in workflow and runs a short baseline session, which sets the first assessment and the default interval.

The duress PIN works everywhere a PIN is asked. It opens a harmless decoy screen, emits `safety.duress`, and the router treats it as Critical without ever showing anything on the device.

4 Workflow B: Court sync

Court dates drive the most important check-ins, so case status is pulled rather than asked from the victim.

- B1

Poll. Nightly for every active CNR, plus a second poll the morning after any scheduled hearing. Source is eCourts case status; an official data-access arrangement is preferred over scraping, which is brittle and must be treated as a prototype-only adapter.
- B2

Diff. Normalise into a snapshot (`stage` , `next_hearing_at` , `court_code` , `disposal`) and compare with the last stored snapshot.
- B3

Emit. `hearing.scheduled` when a new date appears, `hearing.completed` when the date passes and a new one is set, `case.transferred` , `case.disposed` .
- B4

Failure handling. Exponential backoff up to 6 attempts. If a case has not synced for 72 hours, flag it stale and create a low-priority task asking the case manager to confirm the next date manually.
- B5

Manual milestones. Investigation milestones that have no reliable public feed (chargesheet filed, relief instalment paid) are entered by the case manager. Entering the FIR date starts configurable milestone timers from a legal-reviewed template.

5 Workflow C: Check-in scheduling

Triggers

Trigger	When	Agenda focus
periodic	Every 7, 14 or 30 days (victim's choice, default 14)	Wellbeing, needs, case updates

pre_hearing	3 days before next_hearing_at	Preparedness, transport, intimidation before court
post_hearing	1 day after hearing	What happened, pressure at court, next date
milestone_due	When a milestone timer expires unmet	Whether the milestone happened
post_referral	3 days after sent, 7 days after delivered	Was the service received and useful
silence_followup	After missed attempts	Gentle re-contact
manual	Case manager requests	Free-text agenda
victim_initiated	Any time, any channel	Whatever the victim raises

Scheduling rules

- **Merge:** triggers within 48 hours of each other merge into one session with a combined agenda, so a victim is never contacted twice in two days.
- **Safe windows:** outbound contact only inside the victim's safe windows. If a trigger falls outside, it moves to the next window.
- **Cool-down:** no scheduled outreach within 24 hours of a completed session, unless the trigger is an escalation follow-up.
- **Neutral wording:** outbound SMS and notifications never mention the case, crime, court or Aavaz by default. Example: "Hello, this is your support service. Reply 1 to talk now or 2 for later."

Long-running workflow (workers, Node.js)

MVP uses BullMQ delayed jobs: each completed check-in enqueues the next one with a computed delay, and new triggers remove and re-add that job. Production moves the same logic to a Temporal workflow per victim, shown here with the JavaScript SDK.

```
// workers/workflows/checkin.js
import { proxyActivities, defineSignal, setHandler, condition, executeChild } from 'temporal/sdk';

const acts = proxyActivities({ startToCloseTimeout: '1 minute', retry: { maximumAttempts: 3 } });

export const wakeSignal = defineSignal('wake'); // trigger added, pause
export const respondedSignal = defineSignal('responded');

export async function victimCheckinWorkflow(victimId) {
```

```

let wake = false;
setHandler(wakeSignal, () => { wake = true; });

while (true) {
  const state = await acts.loadLifecycle(victimId);
  if (['OPTED_OUT', 'CLOSED', 'PURGED'].includes(state)) return;
  if (state === 'PAUSED') { wake = false; await condition(() => wake); continue;

  const next = await acts.computeNextTrigger(victimId);          // merges trigger
  wake = false;
  const interrupted = await condition(() => wake, Math.max(next.at - Date.now(), 0));
  if (interrupted) continue;                                     // something closed

  const outcome = await executeChild(runOutreach, { args: [victimId, next] });
  if (outcome === 'no_response') await executeChild(silenceLadder, { args: [victimId] });
}

export async function silenceLadder(victimId) {
  let replied = false;
  setHandler(respondedSignal, () => { replied = true; });

  await acts.attempt(victimId, 'preferred');
  if (await condition(() => replied, '24 hours')) return;
  await acts.attempt(victimId, 'alternate');
  if (await condition(() => replied, '72 hours')) return;

  const last = await acts.lastAssessment(victimId);
  const priority = last.level ≥ 2 || last.openThreat ? 'serious' : 'okay';
  await acts.createTask(victimId, { type: 'silence', priority });
}

```

6 Workflow D: Live conversation

Channel adapters and envelope

The Node `channel-gateway` holds every victim connection. Text (app and web) runs over Socket.IO. SMS uses a reduced menu flow with numbered choices. Voice audio goes to `speech-svc` for streaming recognition, the text goes to the same agent, and replies come back through text-to-speech with barge-in so the victim can interrupt. The gateway validates every turn with Zod into one envelope:

```

{
  "session_id": "ses_01J...",

```

```
"victim_id": "vic_01J...",
"channel": "voice|text|sms",
"language": "ta-IN",
"turn_index": 7,
"text": "normalised user text",
"asr_confidence": 0.82,
"received_at": "2026-09-16T11:04:22+05:30"
}
```

Gateway relay (channel-gateway, Node.js)

```
// channel-gateway/src/session.gateway.js
socket.on('turn', async (msg) => {
  const envelope = EnvelopeSchema.parse({ ...msg, sessionId, victimId, channel:
  const res = await fetch(`${AGENT_URL}/v1/sessions/${sessionId}/turns`, {
    method: 'POST',
    headers: {
      'content-type': 'application/json',
      authorization: `Bearer ${await serviceToken('agent-svc')}`,
      'x-request-id': envelope.turnId,
    },
    body: JSON.stringify(envelope),
    signal: AbortSignal.timeout(20_000),
  });

  for await (const evt of parseSSE(res.body)) {
    if (evt.event === 'token') socket.emit('token', evt.data);
    if (evt.event === 'safety') socket.emit('crisis_resources', JSON.parse(evt.d
    if (evt.event === 'done') socket.emit('turn_end');
  }
});
```

Context assembly (memory-svc, Python)

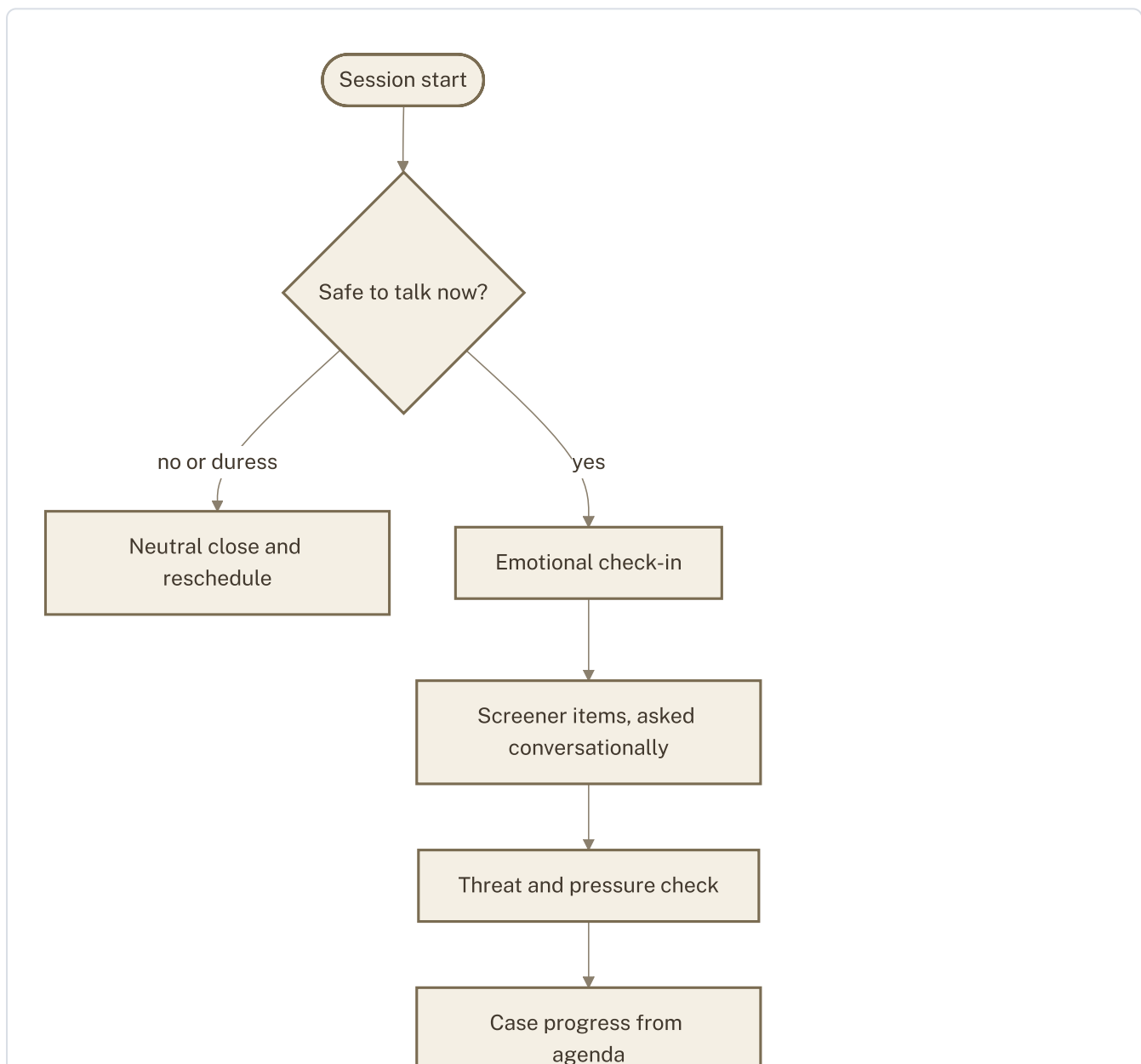
The memory service builds the prompt context once per session and refreshes retrieval per turn. Budget in tokens is shown for a 16k context model.

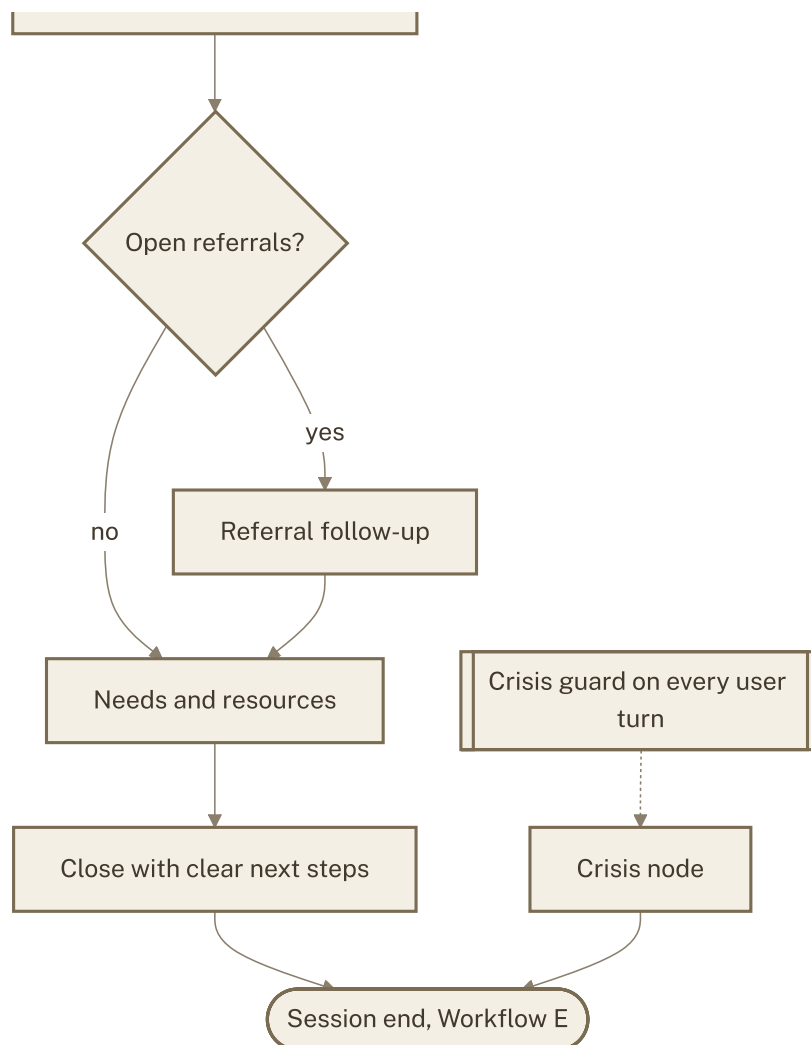
Block	Source	Budget
Policy prompt	Versioned system prompt: trauma-informed rules, no legal advice, no diagnosis, no promises, crisis protocol	1,500
Case state card	Structured fields: stage, next hearing, open referrals and their states, last level and trend, open flags	600

Rolling summary	case_summaries	500
Session agenda	Merged triggers from Workflow C	300
Retrieved memories	pgvector top-8 by cosine, filtered WHERE victim_id = \$1 , MMR re-rank to 4	1,600
Recent turns	Current session	remaining

Retrieval must be victim-scoped in the SQL itself and enforced again by row-level security. The retrieval function takes no free-form filter parameter, so no prompt or bug can widen it.

Conversation graph





Turn endpoint (agent-svc, Python)

```

@app.post("/v1/sessions/{session_id}/turns")
async def post_turn(session_id: str, env: Envelope, _=Depends(verify_service_jwt)):
    async def events():
        async for kind, data in handle_turn_stream(session_id, env): # yields
            yield {"event": kind, "data": data}
    return EventSourceResponse(events())
  
```

Turn loop with crisis guard (agent-svc, Python)

```

async def handle_turn(session, envelope):
    # 1. fast safety check runs before the agent sees the turn
    risk = crisis_rules.match(envelope.text, session.language) # lexicons
    if not risk:
        risk = await crisis_classifier.score(envelope.text) # small model
    if risk.level == CRITICAL:
        await bus.emit("safety.crisis_detected", session.ref, risk) # pages on-
        return await graph.jump(session, "crisis")
  
```

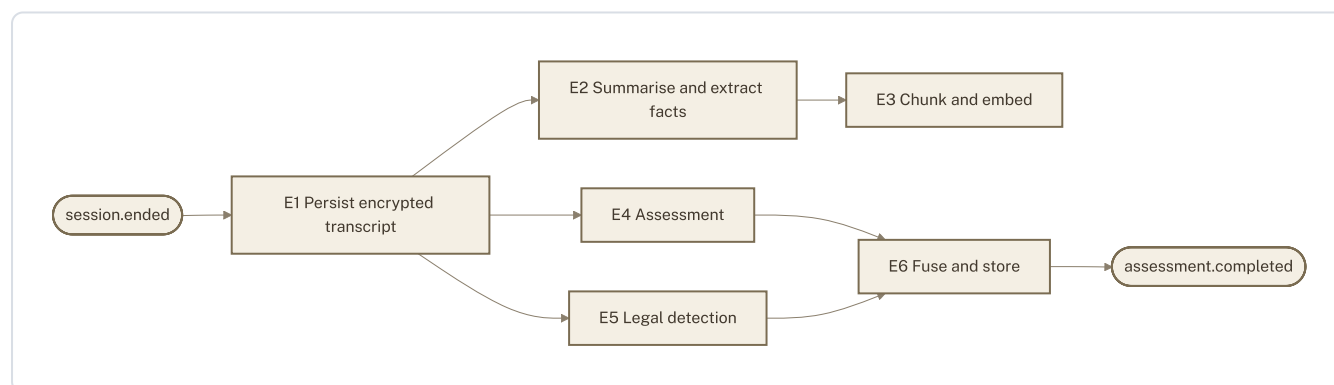
```
# 2. normal graph step
reply = await graph.step(session, envelope)

# 3. output guard before anything is sent
verdict = output_guard.check(reply)    # legal-advice claims, promises, blame
if not verdict.ok:
    reply = await graph.regenerate(session, constraints=verdict.reasons)
return reply
```

The crisis node shows emergency numbers (112, Tele-MANAS 14416, Women Helpline 181 where relevant), keeps the person engaged in calm language, asks whether they are in a safe place, and offers an immediate callback from the on-call case manager. Whether the system may contact emergency services without explicit consent is a policy decision for the operating agency and its legal advisors; the MVP only contacts the on-call human.

7 Workflow E: Post-session pipeline

Runs asynchronously when a session ends or times out after 30 minutes of inactivity. A Node worker orchestrates it and stores the results; the Python services only compute. It never replaces the live crisis guard; it adds the slower, deeper analysis.



E1 Persist (Node). The worker fetches the transcript once from `agent-svc`, encrypts it with a per-victim data key and writes it to object storage; audio kept only if `voice_recording` consent exists, with 90-day retention.

E2 Summarise (memory-svc). LLM produces a session summary (under 200 words) and updates the rolling summary. It also extracts structured facts: new threats (who by role, when, what), needs raised, service experiences. Extracted facts are marked `unverified` until a case manager confirms them.

E3 Embed (memory-svc). Chunk by sliding windows of 4 turns with 1-turn overlap,

embed with a multilingual model (1,024 dimensions), store with `victim_id`, `session_id`, `created_at`.

- E4 Assessment (analysis-svc).** Three independent signals: screener score, LLM classifier, safety rules. See below.
- E5 Legal detection (analysis-svc).** Multi-label classification over the issue taxonomy, then statute retrieval and citation verification.
- E6 Store (Node).** The worker writes the assessment and extracted facts through Prisma and emits `assessment.completed`.

```
// workers/postSession.js (BullMQ flow: children run first, the parent stores results)
await flowProducer.add({
  name: 'store-assessment',
  queueName: 'post-session',
  data: { sessionId, victimId },
  children: [
    { name: 'persist-transcript', queueName: 'post-session', data: { sessionId } },
    { name: 'memory-ingest', queueName: 'ai-calls', data: { sessionId, path: '/v...' } },
    { name: 'assess', queueName: 'ai-calls', data: { sessionId, path: '/v...' } },
    { name: 'legal-detect', queueName: 'ai-calls', data: { sessionId, path: '/v...' } },
  ],
});

// parent processor
new Worker('post-session', async (job) => {
  if (job.name !== 'store-assessment') return handleChild(job);
  const results = await job.getChildrenValues();
  const assessment = pick(results, 'assess');
  const legal = pick(results, 'legal-detect');
  const row = await prisma.assessment.create({ data: toAssessmentRow(job.data, assessment, legal) });
  await bus.publish('assessment.completed', { assessmentId: row.id, victimId: row.victimId, finalLevel: row.finalLevel, reviewRequired: row.reviewRequired });
});
```

Distress levels

Good **Okay** **Bad** **Serious** **Critical** — Critical is the internal fifth level for imminent danger, separate from the four levels shown to staff as distress.

LLM classifier output (structured)

```
{
```

```

    "level": "bad",
    "confidence": 0.71,
    "dimensions": {
        "emotional_distress": 3,          // 0-4
        "safety_threat": 2,
        "functional_impact": 2,
        "support_access": 1
    },
    "risk_flags": ["intimidation_reported", "sleep_disruption"],
    "evidence": [
        {"turn_index": 9, "span": "they came to our street again", "supports": "safe"},
    ],
    "needs": ["legal_aid", "counselling"],
    "review_required": false
}

```

Fusion logic (analysis-svc, Python)

Node sends the previous level and whether an unresolved threat is open with the `/v1/analysis/assess` request, so the Python side stays stateless.

```

LEVEL = {"good":0, "okay":1, "bad":2, "serious":3, "critical":4}

def screener_level(phq4_total):
    # PHQ-4 bands: 0-2, 3-5, 6-8, 9-12
    return 0 if phq4_total ≤ 2 else 1 if phq4_total ≤ 5 else 2 if phq4_total ≤ 8 else 3

def fuse(screener, llm, rules, previous, open_threat):
    candidates = [rules.level, llm.level]
    if screener is not None:
        candidates.append(screener_level(screener))
    final = max(candidates)
    review = False

    if llm.confidence < 0.6:
        # uncertain model: never downgrade
        review = True
    if previous is not None and final - previous ≥ 2:
        final, review = max(final, LEVEL["bad"]), True
    if open_threat:
        # an unresolved threat keeps a flow
        final = max(final, LEVEL["bad"])
    return final, review

```

Legal issue taxonomy

Code

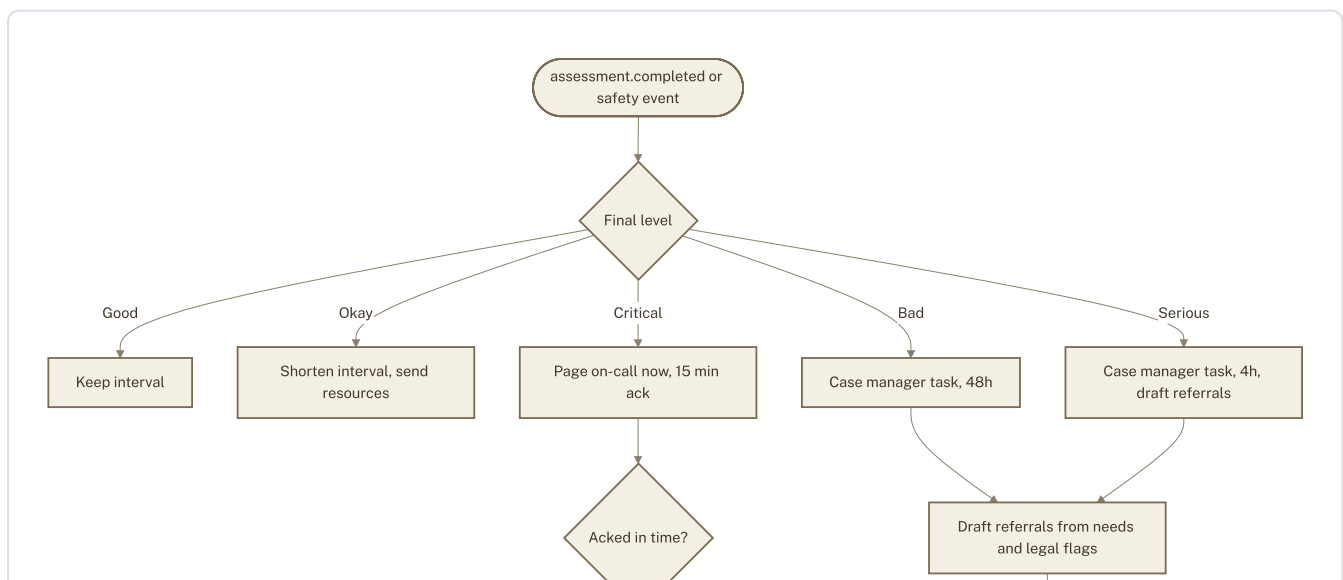
Signal in dialogue

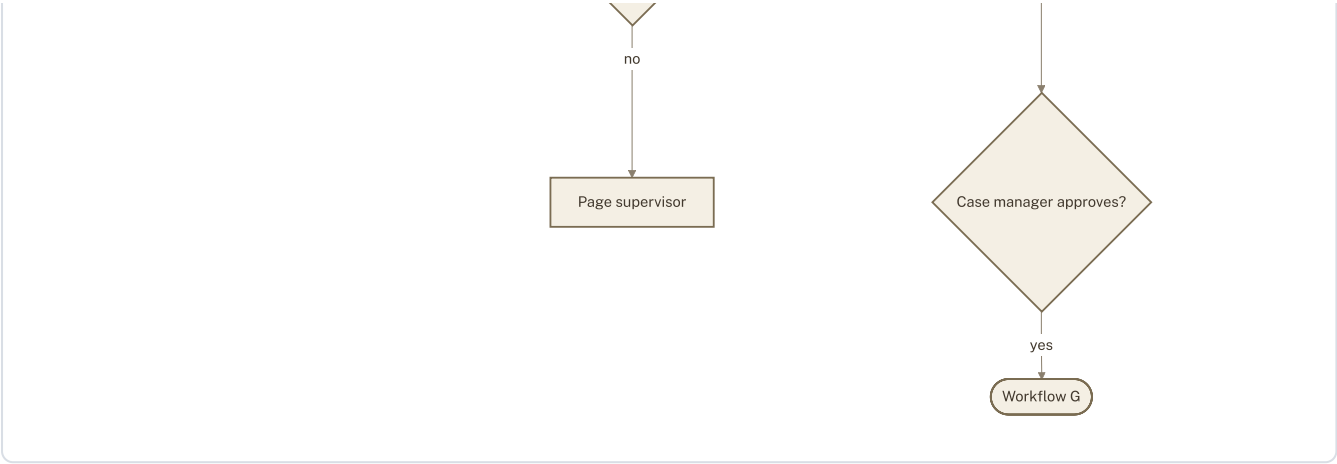
Reference to verify

INTIMIDATION	Threats to withdraw complaint or change testimony	Witness Protection Scheme; BNSS s.398; PoA Act s.15A
COMPROMISE_PRESSURE	Pressure or offers to "settle"	PoA Act s.15A; BNS intimidation provisions
POLICE_INACTION	Refusal to register, record statement or act	BNSS FIR and investigation provisions
INVESTIGATION_DELAY	No chargesheet past configured window	PoA Rules r.7; BNSS s.193 (specified offences)
RELIEF_NOT_PAID	Compensation or relief instalment not received	PoA Rules r.12 schedule; BNSS s.396
HEARING_NOT_INFORMED	Victim not told about hearings or bail proceedings	PoA Act s.15A
IDENTITY_DISCLOSURE	Name or photo of sexual-offence survivor published	BNS s.72
SOCIAL_BOYCOTT	Denial of access, work, water or services after complaint	PoA Act s.3

Citation verification: a citation is kept only if its chunk ID appears in the retrieval results for that issue; otherwise it is dropped and the issue is shown as "reference needed". Legal flags are always labelled "possible issue for legal review" and are never shown to the victim as conclusions.

8 Workflow F: Triage and routing





Level	Automatic actions	Human SLA	Lifecycle
Good	Keep victim's interval	None	MONITORING
Okay	Next periodic check-in at half interval (min 7 days); share self-help resources	None	MONITORING
Bad	Create task; draft referrals for needs and legal flags	Contact within 48 h	ESCALATED
Serious	Create high-priority task; draft referrals; notify supervisor dashboard	Call-back within 4 h	ESCALATED
Critical	Page on-call by SMS and push; crisis resources already shown in session	Ack within 15 min, contact within 30 min	ESCALATED

Routing rules (triage-router module, Node.js)

Rules are stored as versioned JSON and evaluated with json-rules-engine. Changing a rule needs approval from a second admin. The duress PIN and live crisis events skip the engine and go straight to the Critical path.

```
// core-api/src/triage/rules.js
engine.addRule({
  name: 'intimidation_before_hearing',
  conditions: { all: [
    { fact: 'legalIssues', operator: 'contains', value: 'INTIMIDATION' },
    { fact: 'daysToHearing', operator: 'lessThanInclusive', value: 7 },
  ]},
  event: { type: 'route', params: { minLevel: 'serious', drafts: ['protection',
}]);

engine.addRule({
  name: 'relief_overdue',
  conditions: { all: [{ fact: 'legalIssues'. operator: 'contains'. value: 'RELIEF
```

```

    event: { type: 'route', params: { minLevel: 'bad', drafts: ['welfare'] } },
  });

  engine.addRule({
    name: 'needs_counselling',
    conditions: { all: [{ fact: 'needs', operator: 'contains', value: 'counselling'
    event: { type: 'route', params: { drafts: ['mental_health'] } },
  });

  export async function route(assessment) {
    const { events } = await engine.run(await buildFacts(assessment));
    const level = events.reduce((lvl, e) => Math.max(lvl, LEVEL[e.params.minLevel]
    const drafts = [...new Set(events.flatMap((e) => e.params.drafts ?? []))];
    return { level, drafts, sla: SLA_BY_LEVEL[level] };
  }

```

Timer escalation (workers, Node.js)

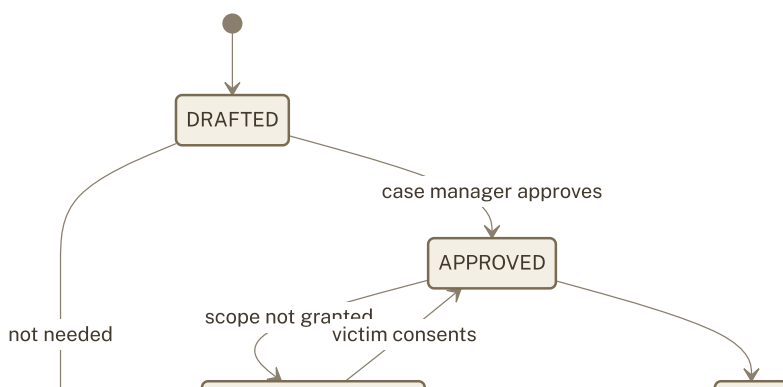
```

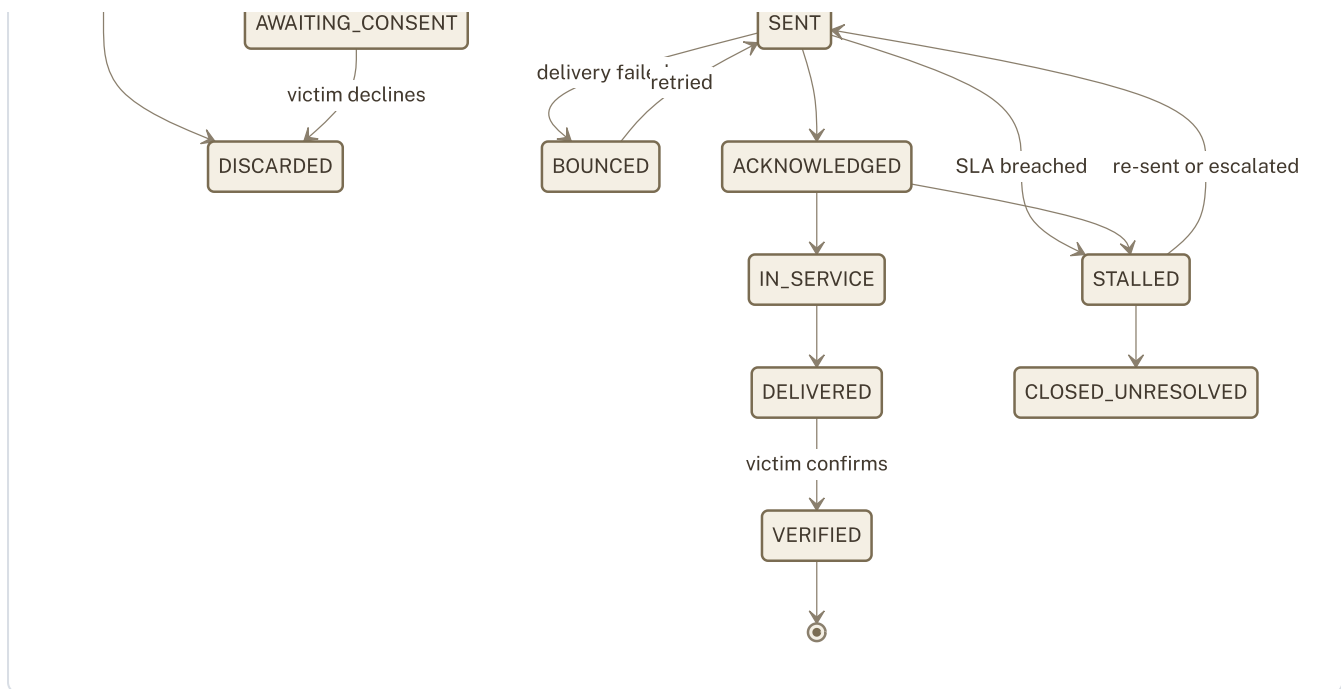
export async function taskSla(taskId, ackMinutes, contactMinutes) {
  let acked = false;
  let contacted = false;
  setHandler(ackSignal, () => { acked = true; });
  setHandler(contactSignal, () => { contacted = true; });

  if (!(await condition(() => acked, `${ackMinutes} minutes`))) {
    await acts.page('supervisor', taskId);
  }
  if (!(await condition(() => contacted, `${contactMinutes} minutes`))) {
    await acts.escalate('district_nodal', taskId);
  }
}

```

9 Workflow G: Referral lifecycle





Minimum necessary data per destination

Destination	Packet contents	Never included
Mental health (Tele-MANAS)	Alias, language, callback number, safe windows, short need summary, current distress level	FIR/CNR, crime details, accused information
Legal aid (DLSA)	Name, CNR/FIR, court, next hearing, legal issue codes with dated summary	Distress scores, counselling notes
Welfare (district social welfare)	Name, FIR, crime category, relief stage pending, dates of earlier applications	Transcripts, mental health data
Protection	Name, case identifiers, dated threat log entries confirmed by case manager	Unverified extracted facts

G1 Consent check. Before `SENT`, the matching `share_*` consent must be granted. Otherwise the ticket moves to `AWAITING_CONSENT` and the case manager asks the victim.

G2 Packet build. A template per destination renders a PDF with Puppeteer in `core-api`; the case manager previews it before sending.

G3 Deliver. MVP adapters: encrypted PDF by email with the password sent by SMS to the receiving officer, or "entered in agency portal manually" with the case manager's confirmation. Future adapters call agency APIs. Every send uses an idempotency key `referral_id:attempt`.

-
- G4** **Track.** Receivers acknowledge via a one-time link (no login, shows nothing but an Acknowledge button and reference number) or the case manager logs acknowledgement after a call.
-
- G5** **SLA.** Default acknowledgement within 3 working days, service start within 10; configurable per destination and district.
-

10 Workflow H: Closed-loop verification

-
- H1** **Ask the victim.** `post_referral` check-ins add agenda items: "Has someone from legal aid contacted you?", "Did the call help?" (helpful, somewhat, not helpful).
-
- H2** **Update ticket.** Yes moves the ticket to `DELIVERED` or `VERIFIED`. No after the SLA moves it to `STALLED`. Not helpful creates a case manager review task.
-
- H3** **Escalation ladder.** Stalled ticket: case manager follow-up, then supervisor after 5 working days, then inclusion in the monthly district nodal report.
-
- H4** **Aggregate.** Nightly job builds anonymised district metrics: referrals by destination and state, median time to service, overdue relief count. Any cell below 5 is suppressed.
-

11 Workflow I: Closure, opt-out and deletion

Case disposed

`case.disposed` moves the victim to `CLOSING`. Two final check-ins at +7 and +30 days cover reaction to the verdict, appeal pressure and pending final relief. The victim then chooses to close or continue for a set period.

Opt-out

Available by app menu, replying STOP, or saying so on a call. The system confirms once, then stops all outbound contact immediately and cancels scheduled workflows. Open referral tickets are closed with a note; receivers are informed the victim withdrew only if the victim agrees.

Deletion job (workers: Node.js)

PurgeJob (workers, helpers,

```
export async function purgeVictim(victimId) {
  if (await hasLegalHold(victimId)) throw new PurgeBlockedError(victimId); // c

  await aiClient.POST('/v1/memory/purge', { body: { victimId } });           // cl
  await storage.deletePrefix(`victims/${victimId}/`);                       //
  await kms.scheduleKeyDeletion(dataKeyId(victimId));                       //

  await prisma.$transaction([
    prisma.assessment.updateMany({ where: { victimId }, data: { victimId: null, s
    prisma.referral.updateMany({ where: { victimId }, data: { victimId: null, pa
    prisma.victim.delete({ where: { id: victimId } }),                        //
  ]);
  await audit.append('victim.purged', { subjectHash: sha256(victimId) });
}
```

Target: purge within 30 days of opt-out or retention expiry. Per-victim data keys make crypto-shredding the backstop for backups.

12 Data model

Suffix `_enc` means field-level encryption with the victim's data key. All victim-linked tables carry `district_code` for row-level security.

Table	Key columns
victims	id, alias_enc, phone_hash, phone_enc, language, district_code, relation_type, preferred_channel, safe_windows (jsonb), checkin_interval_days, lifecycle_state, assigned_cm_id, created_at
cases	id, victim_id, cnr, fir_no_enc, police_station, fir_year, crime_category, court_code, stage, next_hearing_at, disposed_at, last_synced_at, sync_stale
milestones	id, case_id, type, due_at, met_at, entered_by
consents	id, victim_id, scope, granted, text_version_hash, channel, captured_by, captured_at
safety_settings	victim_id, duress_pin_hash, disguise_enabled, safe_word_hash, trusted_contact_enc
checkins	id, victim_id, triggers (text[]), scheduled_for, status, channel, attempt_no,

	session_id
sessions	id, victim_id, channel, language, started_at, ended_at, transcript_uri, audio_uri, crisis_triggered
memory_chunks	id, victim_id, session_id, text_enc, embedding vector(1024), created_at
case_summaries	victim_id, summary_enc, version, updated_at
extracted_facts	id, victim_id, session_id, type, detail_enc, verified_by, verified_at
assessments	id, session_id, victim_id, screener_total, llm_level, llm_confidence, rules_level, final_level, dimensions (jsonb), risk_flags (text[]), legal_issues (jsonb), review_required, model_version, prompt_version
tasks	id, victim_id, type, priority, assignee_id, sla_due_at, acked_at, completed_at, status
referrals	id, victim_id, destination_type, destination_org_id, status, packet_uri, sent_at, acked_at, delivered_at, verified_at, sla_due_at, helpfulness, idempotency_key
staff	id, org_id, role, district_scope (text[]), languages, caseload_cap, on_call_schedule
audit_log	id, actor_id, action, resource_type, resource_id, reason, at, prev_hash, hash

Node-owned tables are defined in `schema.prisma`; `core-api` sets `app.district_scope` on each request's connection before queries. Python-owned tables (`memory_chunks` , `case_summaries` , `statute_chunks`) are created with Alembic migrations, and the vector query below runs inside `memory-svc` .

```
-- retrieval is always victim-scoped and indexed per partition
CREATE INDEX ON memory_chunks USING hnsw (embedding vector_cosine_ops);
CREATE POLICY cm_scope ON victims
  USING (district_code = ANY (current_setting('app.district_scope')::text[]));

SELECT id, text_enc, 1 - (embedding ⇔ $2) AS score
FROM memory_chunks
WHERE victim_id = $1
ORDER BY embedding ⇔ $2
LIMIT 8;
```

15 Event Catalog

Events carry IDs and codes, never free-text victim content. Consumers fetch details through authorised service calls.

Event	Producer	Consumers	Payload keys
victim.registered	profile-case	scheduler, audit	victim_id, district_code, channel
consent.changed	identity-consent	referral, memory, audit	victim_id, scope, granted
case.identifier.added	profile-case	court-sync	case_id, cnr
hearing.scheduled / .completed	court-sync	scheduler	case_id, hearing_at
case.disposed	court-sync	scheduler, profile-case	case_id, disposed_at
checkin.due	scheduler	channel-gateway	checkin_id, victim_id, channel, triggers
session.ended	conversation-agent	memory, assessment, legal-detect	session_id, victim_id, crisis_triggered
safety.crisis_detected	conversation-agent	triage-router	session_id, victim_id, rule_id
safety.duress	identity-consent	triage-router	victim_id, channel
assessment.completed	assessment	triage-router, console	assessment_id, victim_id, final_level, review_required
task.created / .breached	triage-router	console, notifier	task_id, priority, sla_due_at
referral.state_changed	referral	scheduler, oversight	referral_id, from, to
victim.opted_out	identity-consent	all services	victim_id

14 API surface

Method and path	Caller	Purpose
POST /v1/auth/otp	Victim app	Request OTP
POST /v1/auth/verify	Victim app	Verify OTP or PIN (duress-aware)
POST /v1/consents	Victim app	Grant or revoke a scope
POST /v1/victims	Victim app, partner staff	Create profile
PATCH /v1/victims/me/preferences	Victim app	Interval, channel, safe windows, pause
POST /v1/cases	Victim app, case manager	Add CNR or FIR
WS /v1/sessions/{id}	Victim app	Live text session
POST /v1/channels/ivr/webhook	Telephony provider	Voice events (signed)
POST /v1/channels/sms/webhook	SMS provider	Inbound SMS (signed)
POST /v1/victims/me/opt-out	Victim app	Stop and schedule purge
GET /v1/console/queue	Case manager	Tasks by priority and SLA
GET /v1/console/victims/{id}	Case manager	Profile, timeline, assessments (audited)
POST /v1/console/assessments/{id}/review	Case manager	Confirm or override level with reason
POST /v1/console/referrals/{id}/approve	Case manager	Approve and send
POST /v1/ack/{token}	Referral receiver	One-time acknowledgement link
GET /v1/oversight/districts/{code}/metrics	District oversight	Anonymised aggregates

Public endpoints above are served by Node (`core-api` and `channel-gateway`). The Python services are reachable only inside the cluster.

Internal AI API (Python, FastAPI)

Method and path	Service	Caller	Purpose
POST <code>/v1/sessions/{id}/turns</code>	agent-svc	channel-gateway	One turn; SSE stream of token, safety and done events
POST <code>/v1/sessions/{id}/end</code>	agent-svc	channel-gateway	Close the graph, emit <code>session.ended</code>
GET <code>/v1/sessions/{id}/transcript</code>	agent-svc	workers	Fetch transcript once for encrypted storage
POST <code>/v1/memory/context</code>	memory-svc	agent-svc	Assemble prompt context for a session
POST <code>/v1/memory/ingest</code>	memory-svc	workers	Summarise, update rolling summary, chunk and embed
GET <code>/v1/memory/summary/{victimId}</code>	memory-svc	core-api	Rolling summary for the console timeline
POST <code>/v1/memory/purge</code>	memory-svc	workers	Delete a victim's chunks, summaries and checkpoints
POST <code>/v1/analysis/assess</code>	analysis-svc	workers	Screeners, classifier, rules and fusion
POST <code>/v1/analysis/legal</code>	analysis-svc	workers	Issue codes with verified citations
WS <code>/v1/speech/stream</code>	speech-svc	channel-gateway	Streaming recognition and synthesis for calls
GET <code>/healthz</code> , <code>/readyz</code>	all	platform	Liveness and readiness

Access by role

Role	Can see	Cannot see
Victim	Own preferences, referral status, resources	Assessment scores, legal flags, staff notes
Case manager	Assigned victims in district scope, transcripts, assessments	Victims outside scope without break-glass
Supervisor	District queue, SLA breaches, overrides	Transcripts unless opened with a logged reason
Referral receiver	Their packet and ack link only	Any system screen
District oversight	Aggregates with small-count suppression	Any individual record
Engineer / admin	Infrastructure, metadata, logs without content	Decrypted victim content

Controls

- **Encryption:** TLS 1.3 everywhere; envelope encryption with a per-victim data key wrapped by a KMS key; keys never leave KMS in plaintext logs.
- **Break-glass:** out-of-scope access needs a typed reason, expires in 2 hours, and notifies the supervisor.
- **Audit:** every read of victim content is logged; each row stores the hash of the previous row so tampering is detectable; daily export to write-once storage.
- **LLM handling:** providers bound by no-training and minimal-retention terms, India-region processing where available; prompts and outputs are logged only as IDs and hashes in general logs.
- **Prompt injection:** victim text is passed as data inside delimited blocks; the agent has no tools that write outside its session or query other victims.
- **Identity protection:** referral packets and dashboards pseudonymise survivors of sexual offences by default.
- **Webhooks:** telephony and SMS webhooks verified by provider signature and replay window.

16 Deployment

Repository layout

```

aavaz/
  apps/
    channel-gateway/      # Node.js, NestJS
    core-api/             # Node.js, NestJS, Prisma
    workers/              # Node.js, BullMQ / Temporal worker
    console/              # React
  ai/
    agent-svc/            # Python 3.12, FastAPI, LangGraph
    memory-svc/           # Python, FastAPI, SQLAlchemy, pgvector
    analysis-svc/         # Python, FastAPI
    speech-svc/           # Python, FastAPI
    shared/               # Pydantic models, JWT verification, eval harness
  contracts/              # exported OpenAPI specs, generated JS clients
  infra/                  # Docker, Helm or Cloud Run configs, Terraform

```

- **Runtime:** Node.js 22 LTS containers for the backend; Python 3.12 containers running Uvicorn for AI services; managed Kubernetes or Cloud Run in an India region; managed Postgres with pgvector and point-in-time recovery.
- **Scaling:** Node services scale on request rate and queue depth; `agent-svc` scales on concurrent streaming sessions; `memory-svc` embeddings can run on a small GPU pool or a hosted embeddings server.
- **Environments:** dev (synthetic data only), staging (synthetic data, production-like), production. Real victim data never enters dev or staging.
- **CI/CD:** Node lint and tests (Vitest), Python lint and tests (Ruff, Pytest), contract check that regenerated JS clients match the committed OpenAPI specs, dependency and container scans, then the model evaluation gate from section 17 for any prompt or model change.
- **Resilience:** target recovery point 15 minutes and recovery time 4 hours; on-call paging uses a second SMS provider as fallback.
- **Degraded mode:** if the Python AI services or LLM provider are down, the Node gateway pauses scheduled check-ins and serves victim-initiated contact from a scripted menu with crisis numbers, plus a case manager callback task. The crisis lexicon is duplicated in the gateway for this path only.

17 Observability and evaluation

Operational metrics

Check-in completion rate, silence ladder hits, time to acknowledge Critical pages, SLA

breach counts, court sync staleness, referral median time to service. Traces via OpenTelemetry with no victim content in spans.

Model evaluation

- **Golden set:** synthetic multilingual dialogues (Tamil, Hindi, English, code-mixed) written with counsellors, labelled for level, flags and legal issues, including adversarial and ambiguous cases.
- **Release gate:** Critical recall at least 0.95 and Serious-or-worse recall at least 0.90 on the golden set; no drop in either versus the current version.
- **Shadow mode:** new model versions score live sessions in parallel for two weeks without affecting routing; differences go to review.
- **Human feedback loop:** case manager overrides with reasons feed a weekly review and the golden set.
- **Fairness checks:** recall compared across language, dialect and caste-related vocabulary slices.